



How to Check if a File or Directory Exists in Bash

Updated Jun 6, 2020 • 5 min read



RELATED TUTORIALS

[Bash: Append to File](#)

[Writing Comments in Bash Scripts](#)

[Pushd and Popd Commands in Linux](#)

[How to Increment and Decrement Variable in Bash \(Counter\)](#)

[Bash Shebang](#)

[How to Check if a String Contains a Substring in Bash](#)

[Bash Source Command](#)

Many times when writing Shell scripts, you may find yourself in a situation where you need to perform an action based on whether a file exists or not.

In Bash, you can use the `test` command to check whether a file exists and determine the type of the file.

[report this ad](#)

The `test` command takes one of the following syntax forms:

```
test EXPRESSION
[ EXPRESSION ]
[[ EXPRESSION ]]
```

If you want your script to be portable, you should prefer using the old `test` `[` command, which is available on all POSIX shells. The new upgraded version of the `test` command `[[` (double brackets) is supported on most modern systems using Bash, Zsh, and Ksh as a default shell.

Check if File Exists

When checking if a file exists, the most commonly used FILE operators are `-e` and `-f`. The first one will check whether a file exists regardless of the type, while the second one will return true only if the FILE is a regular file (not a directory or a device).

The most readable option when checking whether a file exists or not is to use the `test` command in combination with the [if statement](#). Any of the snippets below will check whether the `/etc/resolv.conf` file exists:

```
FILE=/etc/resolv.conf
if test -f "$FILE"; then
    echo "$FILE exists."
fi
```

```
FILE=/etc/resolv.conf
if [ -f "$FILE" ]; then
    echo "$FILE exists."
fi
```

```
FILE=/etc/resolv.conf
if [[ -f "$FILE" ]]; then
```

```
    echo "$FILE exists."
fi
```

If you want to perform a different action based on whether the file exists or not simply use the if/then construct:

```
FILE=/etc/resolv.conf
if [ -f "$FILE" ]; then
    echo "$FILE exists."
else
    echo "$FILE does not exist."
fi
```

Always use [double quotes](#) to avoid issues when dealing with files containing whitespace in their names.

You can also use the test command without the if statement. The command after the && operator will only be executed if the [exit status](#) of the test command is true,

```
test -f /etc/resolv.conf && echo "$FILE exists."
```

```
[ -f /etc/resolv.conf ] && echo "$FILE exists."
```

```
[[ -f /etc/resolv.conf ]] && echo "$FILE exists."
```

If you want to run a series of command after the && operator simply enclose the commands in curly brackets separated by ; or &&:

```
[ -f /etc/resolv.conf ] && { echo "$FILE exist."; cp "$FILE" /tmp; }
```

Opposite to &&, the statement after the || operator will only be executed if the exit status of the test command is false.

```
[ -f /etc/resolv.conf ] && echo "$FILE exist." || echo "$FILE does not exist."
```

Check if Directory Exist

The operators -d allows you to test whether a file is a directory or not.

For example to check whether the /etc/docker directory exist you would use:

```
FILE=/etc/docker
if [ -d "$FILE" ]; then
    echo "$FILE is a directory."
fi
```

```
[ -d /etc/docker ] && echo "$FILE is a directory."
```

You can also use the double brackets [[instead of a single one [.

Check if File does Not Exist

Similar to many other languages, the test expression can be negated using the ! (exclamation mark) logical not operator:

```
FILE=/etc/docker
if [ ! -f "$FILE" ]; then
    echo "$FILE does not exist."
```

```
fi
```

Same as above:

```
[ ! -f /etc/docker ] && echo "$FILE does not exist."
```

Check if Multiple Files Exist

Instead of using complicated nested if/else constructs you can use `-a` (or `&&` with `[]`) to test if multiple files exist:

```
if [ -f /etc/resolv.conf -a -f /etc/hosts ]; then
    echo "Both files exist."
fi

if [[ -f /etc/resolv.conf && -f /etc/hosts ]]; then
    echo "Both files exist."
fi
```

Equivalent variants without using the IF statement:

```
[ -f /etc/resolv.conf -a -f /etc/hosts ] && echo "Both files exist."

[[ -f /etc/resolv.conf && -f /etc/hosts ]] && echo "Both files exist."
```

File test operators

The test command includes the following FILE operators that allow you to test for particular types of files:

- `-b FILE` - True if the FILE exists and is a special block file.
- `-c FILE` - True if the FILE exists and is a special character file.
- `-d FILE` - True if the FILE exists and is a directory.
- `-e FILE` - True if the FILE exists and is a file, regardless of type (node, directory, socket, etc.).
- `-f FILE` - True if the FILE exists and is a regular file (not a directory or device).
- `-G FILE` - True if the FILE exists and has the same group as the user running the command.
- `-h FILE` - True if the FILE exists and is a symbolic link.
- `-g FILE` - True if the FILE exists and has set-group-id (`sgid`) flag set.
- `-k FILE` - True if the FILE exists and has a sticky bit flag set.
- `-L FILE` - True if the FILE exists and is a symbolic link.
- `-O FILE` - True if the FILE exists and is owned by the user running the command.
- `-p FILE` - True if the FILE exists and is a pipe.
- `-r FILE` - True if the FILE exists and is readable.
- `-S FILE` - True if the FILE exists and is a socket.
- `-s FILE` - True if the FILE exists and has nonzero size.
- `-u FILE` - True if the FILE exists, and set-user-id (`suid`) flag is set.
- `-w FILE` - True if the FILE exists and is writable.
- `-x FILE` - True if the FILE exists and is executable.

Conclusion

In this guide, we have shown you how to check if a file or directory exists in Bash.

If you have any questions or feedback, feel free to leave a comment.

[bash](#) [terminal](#)



[report this ad](#)

If you like our content, please consider buying us a coffee.
Thank you for your support!



Sign up to our newsletter and get our latest tutorials and news
straight to your mailbox.

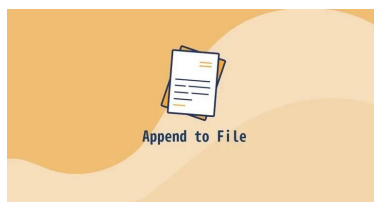
[Subscribe](#)

We'll never share your email address or spam you.

Related Articles

DEC 21, 2019

[Bash: Append to File](#)



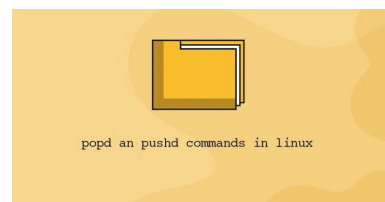
OCT 2, 2019

[Writing Comments in Bash Scripts](#)



SEP 21, 2019

[Pushd and Popd Commands in Linux](#)



[Show comments \(9\)](#)